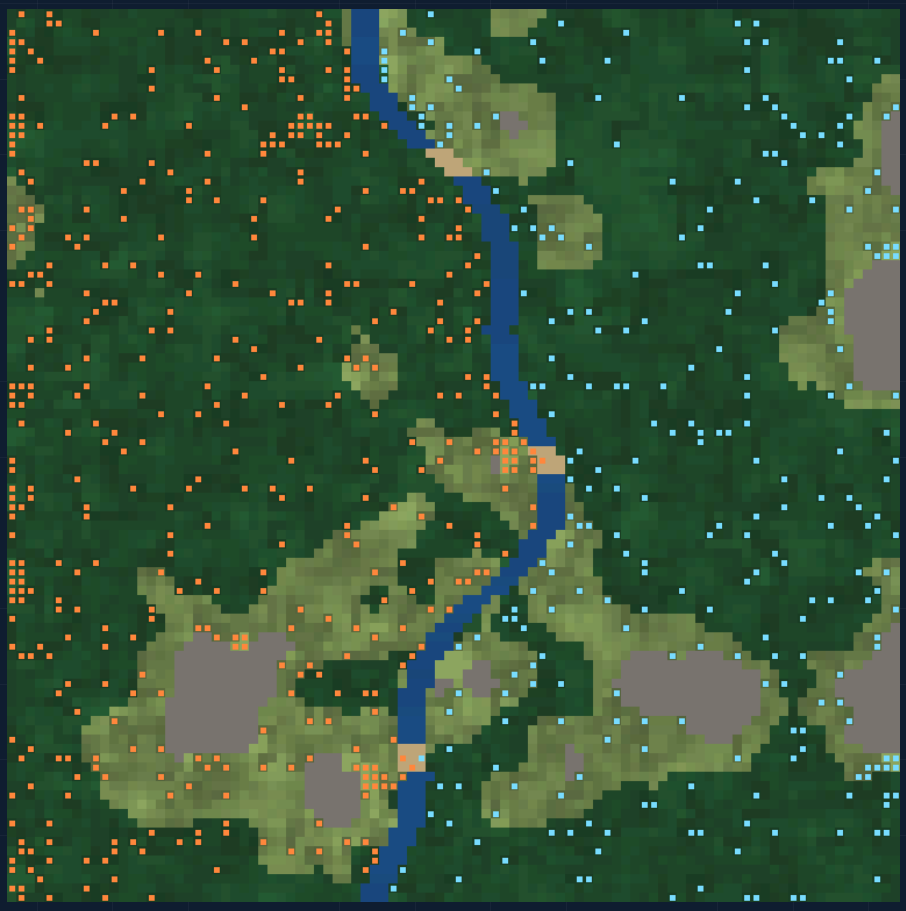
SETUP PAPER · SCENARIOS A + B

River World on the Desktop

Scenarios A and B in one sealed world, with a river down the middle

11 OCTOBER 2026



COVERS

Install · four first-run checks · sixteen seeds on a Ryzen 9 5950X · the first experiment · six more · reading the log · the RTX 3070 · a week-long run

BUILT ON

Running Persistent AI Worlds, second edition

SOURCES

7 numbered references

PREPARED FOR

Nerds489

About this paper

The research paper *Running Persistent AI Worlds* (second edition, 10 October 2026) ran two scenarios on a laptop: two learning agents in a forest that never ends (Scenario A), and an evolving population sealed in a box with a fixed amount of energy (Scenario B). This paper joins them into one world, adds a river, and sets it up to run for days on the Ryzen 9 5950X desktop.

- **A new simulation, tested.** `river_world.py` puts Scenario A's learning agents inside Scenario B's sealed, evolving population, on a cylinder of land split by a river that carries nutrients and can be crossed only at three fords. Energy is conserved and checked every tick; a stopped run resumes exactly where it left off (Appendix B).
- **A setup for sixteen cores.** One process per core, one seed per process, a launcher, a summariser and a map tool (Sections 3 to 5).
- **A first experiment, measured.** Does it matter whether a child inherits what its parent learned? Four seeds of each, 300 simulated years, run on the laptop (Section 6).
- **What to do next.** Six experiments, one change each, and how to read the log (Sections 7 and 8); the RTX 3070's place in all this (Section 9); and a checklist for a week-long run (Section 10).

Measured numbers come from the laptop the paper was built on; numbers for the desktop are labelled as estimates.

Contents

1. The desktop
2. River world: Scenarios A and B in one world
3. Install
4. First run: four checks
5. Running sixteen seeds
6. The first experiment: should learning be inherited?
7. Experiments to run next
8. Reading the log
9. Using the RTX 3070
10. Before a week-long run - Appendix A: Parameters - Appendix B: How the kit was tested - References

30,000
energy units in the world, never created or destroyed, checked 400 times a simulated year

3 fords
the only way across the river; the world wraps north to south and is walled east to west

16 seeds
at once on the 5950X: one process per physical core, about 80 MB each

0.18 vs 0.05
learning rate after 300 years with learning inherited, and without: evolution turns learning down when it is not passed on (Section 6)

70 → 21
energy needed to reproduce, at the start and after 300 years: evolution chose many small, hungry agents (Section 6)

identical
a stopped and resumed run matches an uninterrupted one, byte for byte (Appendix B)

1 The desktop

This paper is written for one machine: the Ryzen desktop running Linux Mint 22.3. Its parts decide how the work is split. The simulation itself is NumPy on the CPU, one process per seed, so the sixteen cores are the engine. The graphics card is for the optional GPU stages in Section 9, and the memory and disks are simply never the constraint.

Table 1. The desktop and what each part does in this setup.

PART	SPECIFICATION	ROLE HERE
CPU	AMD Ryzen 9 5950X, 16 cores / 32 threads, boost to 4.9 GHz	Sixteen seeds at once, one per physical core
Memory	64 GB DDR4-3200 (2 × 32 GB, dual rank)	About 80 MB per seed: sixteen seeds use about 1.3 GB
GPU	NVIDIA GeForce RTX 3070 LHR, 8 GB	Optional: JAX environments and a local language model (Section 9)
Motherboard	Gigabyte B550M DS3H AC	
Operating system	Linux Mint 22.3 (Ubuntu 24.04 base), kernel 6.17	Python 3.12 from the distribution

Three consequences follow.

1. **Seeds, not threads.** One river world is a single-threaded Python loop over NumPy arrays. It cannot use sixteen cores for one world, but sixteen independent worlds use them perfectly. That suits the science too: a result that holds in one seed is an anecdote; one that holds in twelve of sixteen is a finding.
2. **Physical cores, not threads.** The 5950X shows 32 logical CPUs, but its second thread per core adds little to this kind of arithmetic. The launcher therefore runs `nproc / 2` = 16 processes, and pins NumPy to one thread each so they do not compete.
3. **Heat, not memory, is the limit for week-long runs.** Sixteen busy cores hold the 5950X at its all-core power limit for days. Watch `sensors` for the first hour (Section 5.3); the chip's limit is 90 °C.

> **Note:** Every timing in this paper was measured on the laptop the paper was built on (Intel Core i7-8665U, 4 cores, 16 GB). Figures for the desktop are estimates until you run the smoke test in Section 4, which takes a minute and replaces them with your own numbers.

2 River world: Scenarios A and B in one world

The research paper ran its two scenarios separately. Scenario A put two learning agents in a forest that never ends; Scenario B sealed an evolving population in a box with a fixed amount of energy. River world joins them: every agent learns during its life, as in A, inside a population that evolves under a fixed energy budget, as in B. A river runs down the middle.

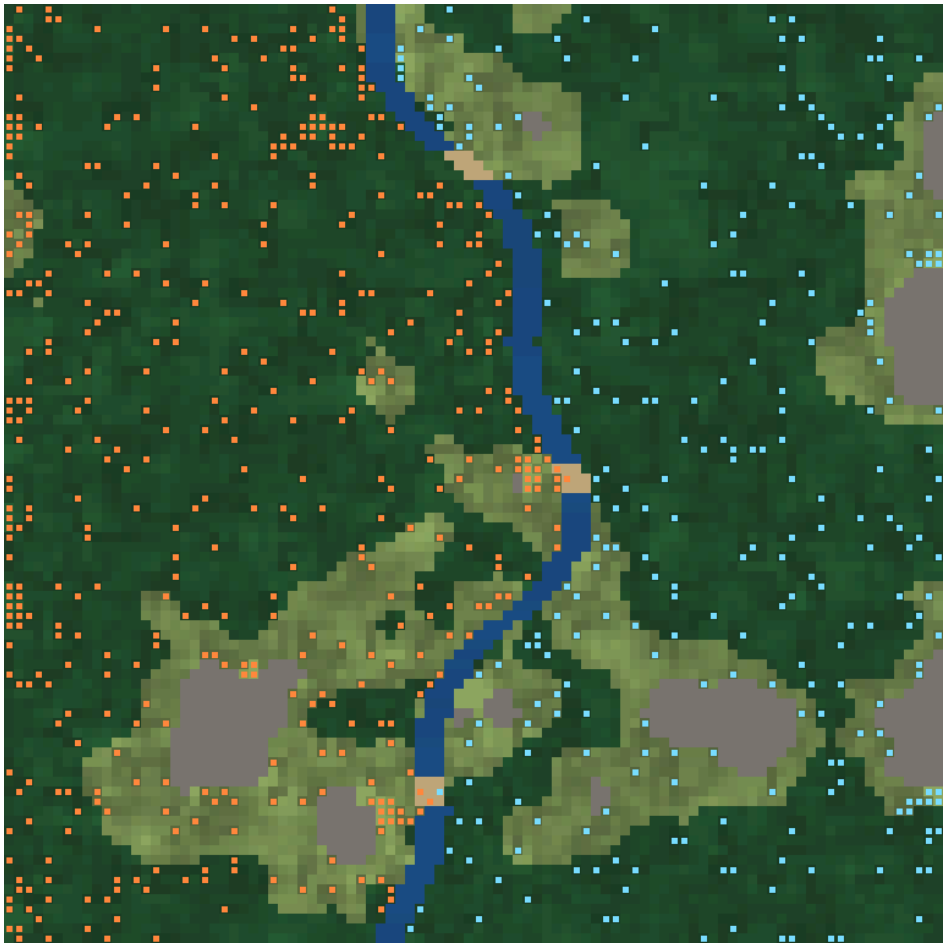


Figure 1. River world after 300 simulated years (seed 1, learning inherited). The river runs north to south and wraps round, so it has no source and no mouth; tan bars are the three fords. Agents are coloured by bank: west orange, east blue. Paler land holds more nutrients; the river darkens where it carries more silt.

2.1 What comes from Scenario A

- **Terrain** from the same smoothed-noise generator: forest, clearings (about 22% of the land) and rock (5%), which nothing can enter.
- **Time.** A day is 20 ticks, the last 40% of it night; a season is 100 ticks; a year is 400. Moving at night costs a quarter more.
- **A 2-bit signal channel.** Every action pairs a move (stay, north, east, south or west) with a signal (silent, 1, 2 or 3), 20 actions in all. An agent hears the loudest signal in the eight cells around it, one tick later. Speaking costs 0.05 energy.
- **Learning in a lifetime.** Each agent is a tabular Q-learner over 240 states: the direction of the richest neighbouring cell (5), its own energy band (3), day or night (2), the signal it hears (4) and whether it is within four cells of the water (2). The reward is the change in its own energy. Ties between untried actions are broken at random.

2.2 What comes from Scenario B

- **A sealed energy budget.** The world holds 30,000 energy units by default, and none is ever created or destroyed. The total is asserted after every tick.
- **Bodies and death.** Resting costs 0.5 a tick and moving 0.9; an agent eats up to 4 a tick from its cell and holds at most 150. It dies of starvation or of old age, and its body becomes detritus where it falls.

- **Heritable genomes.** Five genes: lifespan, the energy needed to reproduce, the mutation scale, the learning rate and the exploration rate. A parent above its threshold splits its energy with a child whose genes are mutated. Evolution can therefore tune *how* agents learn, which is the Baldwin effect that Hinton and Nowlan demonstrated in 1987 [1].
- **What the child learns from the parent** is a switch, `--inherit` : `copy` hands over the parent's Q-table (learning passes down, Lamarckian in effect), `half` hands it over at half strength, and `none` starts every child blank, so only the genes for learning are inherited.

2.3 What the river adds

- **Two banks.** The world wraps north to south but has walls east and west, so it is a cylinder and the river cuts it in two. The channel is three cells wide and meanders up to about 11 cells either side of the centre line. Agents can cross only at three fords, at rows 16, 48 and 80.
- **A moving store of nutrients.** Each tick the river erodes 0.2% of the nutrients on its banks, carries a fifth of each stretch's load one row downstream, and lays 1% back down as silt on the banks. Because the river is a closed loop, everything it carries comes back.
- **Seasons and events.** Silt deposition rises two and a half times in spring. At the start of each season the world may roll an event: a spring flood (one year in four: six times the silt, three times the erosion, for 40 ticks), a summer drought (about one in seven: the river slows to a quarter and drops less silt), or a fire (a 12% chance at the start of each summer and autumn: a circle of radius 6 burns, nutrients become detritus, and agents caught in it lose half their energy). `--no-events` switches them off.

2.4 The energy ledger

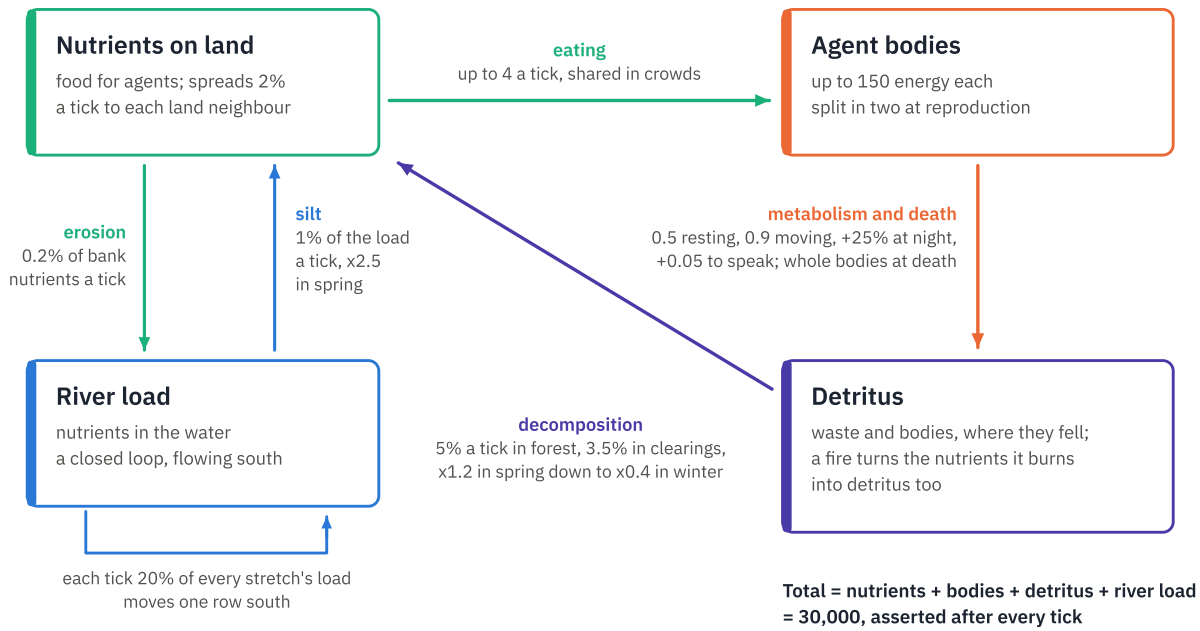


Figure 2. Where the energy goes. Four pools hold all 30,000 units; arrows are the only ways energy moves between them. Events change the rates on the arrows, never the total.

Decomposition turns detritus back into nutrients in place, at 5% a tick in forest, 3.5% in clearings and 2% on fords, and seasonally: a fifth faster in spring, under half as fast in winter. Nutrients also spread 2% a tick to each neighbouring land cell, never into rock or deep water. The full parameter list is in Appendix A.

> **Key insight:** Because nothing is created, every pattern in the population has to be paid for out of the same 30,000 units. Rich banks mean poorer land somewhere else; a boom on one side of the river is energy that is not available on the other.

3 Install

Everything in this section runs in a terminal on the desktop. The river world needs only Python 3.10 or newer and NumPy; the GPU stages in Section 9 have their own environments so that nothing here can break them.

3.1 The simulation environment

```
sudo apt update && sudo apt install -y python3.12-venv lm-sensors tmux
python3 -m venv ~/venvs/river && source ~/venvs/river/bin/activate
pip install -U pip numpy
python3 -c "import numpy, sys; print(sys.version.split()[0], numpy.__version__)"
```

Use the virtual environment rather than the system Python. On the laptop this paper was built on, a NumPy 2 in the user site directory shadowed the distribution's matplotlib, which was built for NumPy 1, and broke it; a venv keeps the two apart. Activate it (`source ~/venvs/river/bin/activate`) in every new terminal before running anything below.

3.2 The files

Copy the `river-world/` folder that comes with this paper to the desktop, for example to `~/river-world`. It holds five files and nothing else:

Table 2. The river world kit.

FILE	WHAT IT DOES
`river_world.py`	The world. One process runs one seed and writes `log.csv` and `checkpoint.pkl` in its run folder
`run_seeds.sh`	Runs many seeds at once, one per physical core, then summarises them
`summarise.py`	One line per seed, the mean and spread across seeds, and a comparison between conditions
`snapshot.py`	Draws a checkpoint as a PNG map (Figure 1 was made with it)
`README.md`	The commands from this paper, in short

```
cd ~/river-world && chmod +x *.py run_seeds.sh
python3 river_world.py --help
```

3.3 Check the graphics driver (only for Section 9)

```
nvidia-smi --query-gpu=name,driver_version,memory.total --format=csv
```

This should name the RTX 3070 with 8,192 MiB. Note the driver version: Section 9 installs the JAX build that matches it.

4 First run: four checks

Before committing sixteen cores to a week, spend ten minutes proving that the world works on this machine. Each check has a pass condition you can see.

4.1 Smoke test and speed

```
cd ~/river-world && source ~/venvs/river/bin/activate
time python3 river_world.py --years 20 --fresh --dir runs/smoke
```

It prints one line, for example:

```
runs/smoke: year 20  population 800 (west 393, east 407)  max generation 21  (0.99 years/s)  energy
conserved to 9.1e-10
```

Pass: the run finishes, both banks are populated, and `energy conserved to` is below 10^{-6} of the 30,000 total. On the laptop this paper was built on, one process ran 0.99 simulated years a second. Write down your own figure; Section 5 uses it.

4.2 Energy is conserved, every tick

There is nothing to do: the check is built in. After every tick the world adds up the nutrients, the detritus, the river's load and every body, and stops with `energy not conserved` if the total has drifted by more than one part in a million. A run that finishes has passed 400 checks a simulated year. The column `energy_drift` in `log.csv` records the drift at each year's end.

4.3 Stop, resume and get the same world

```
python3 river_world.py --years 6 --fresh --seed 5 --dir runs/straight
python3 river_world.py --years 20 --fresh --seed 5 --dir runs/stopped &
sleep 4; kill -INT %1; wait          # the same as pressing Ctrl-C
python3 river_world.py --years 6 --dir runs/stopped      # carries on from where it stopped
head -n 7 runs/straight/log.csv | cmp - <(head -n 7 runs/stopped/log.csv) && echo identical
```

Ctrl-C (or `kill`, or a shutdown) makes the world finish its current year, save, and exit. A checkpoint holds the whole world, including the random number generator, so a resumed run follows exactly the path an uninterrupted one would have. **Pass:** `identical`. If the machine loses power instead, the next start trims any log rows written after the last checkpoint (every 10 years by default; `--save-every 1` for every year) and carries on from there.

4.4 Look at it

```
python3 snapshot.py runs/smoke                    # agents coloured by bank
python3 snapshot.py runs/smoke --colour signal    # by each agent's favourite signal
xdg-open runs/smoke/map-year-20-side.png
```

Pass: a map like Figure 1: green land, a blue river down the middle, three tan fords, and agents on both banks.

5 Running sixteen seeds

5.1 The launcher

```
./run_seeds.sh 16 1000 main                          # 16 seeds x 1,000 years into runs/main/
./run_seeds.sh 16 1000 darwin --inherit none         # a second condition, its own folder
./run_seeds.sh 16 1000 main                          # again: every seed carries on another 1,000
years
```

The arguments are the number of seeds, the years to run, a name for the condition, and any `river_world.py` options. The launcher runs one process per physical core (16 on the 5950X), keeps NumPy single-threaded inside each, lowers their priority with `nice` so the desktop stays usable, and appends each seed's output to `runs/<name>/seed-N.out`. When every seed has finished it prints the summary from Section 6. Running it again resumes every seed; options such as `--inherit` only take effect when a seed's world is first created, and the world says so if you pass a different one later.

5.2 How long it takes

Table 3. Throughput, measured on the laptop and estimated for the desktop.

	LAPTOP (I7-8665U, MEASURED)	DESKTOP (5950X, ESTIMATE)
One process, about 800 agents	0.99 years/s	about 1.4 years/s
All cores busy	0.32 years/s per process, 8 processes on 4 cores	about 1.1 years/s per process, 16 processes
16 seeds × 1,000 years	about 1.7 hours (8 at a time)	about 15 minutes (all 16 at once)
16 seeds for a week		about 700,000 simulated years each

The desktop column assumes each 5950X core is about 1.4 times as fast as one of the laptop's on this single-threaded NumPy work, and that sixteen processes on sixteen cores lose about a fifth of that to shared memory bandwidth and lower all-core clocks. Treat it as a guess until Section 4.1 has given you the real one-process figure; the all-cores figure is the one to measure next, by running `./run_seeds.sh 16 20 speedtest` and reading the `years/s` on each line.

Population sets the pace: the work per tick grows with the number of agents. Doubling `--energy` roughly doubles the population and halves the speed.

5.3 Heat and power

A week of sixteen busy cores is a sustained load the 5950X will hold at its power limit. In the first hour:

```
watch -n 5 "sensors | grep -E 'Tctl|Tccd'"
```

Under 85 °C is comfortable; the chip throttles itself at 90 °C, which slows the runs but harms nothing. If it runs hot, use `JOBS=12 ./run_seeds.sh ...` to leave four cores idle. The runs do not care about the GPU, so it can be used for Section 9 at the same time.

5.4 Surviving the terminal

Run the launcher inside `tmux` so that closing the terminal or logging out does not stop it:

```
tmux new -s river           # start a session, then run ./run_seeds.sh inside it
# detach with Ctrl-b then d; reattach later with: tmux attach -t river
```

A reboot stops the runs cleanly (each one saves before it exits). Afterwards, run the same `./run_seeds.sh` line again and every seed resumes. Keep the kit and its `runs/` folder in your home directory, never under `/tmp`, which is emptied at every boot.

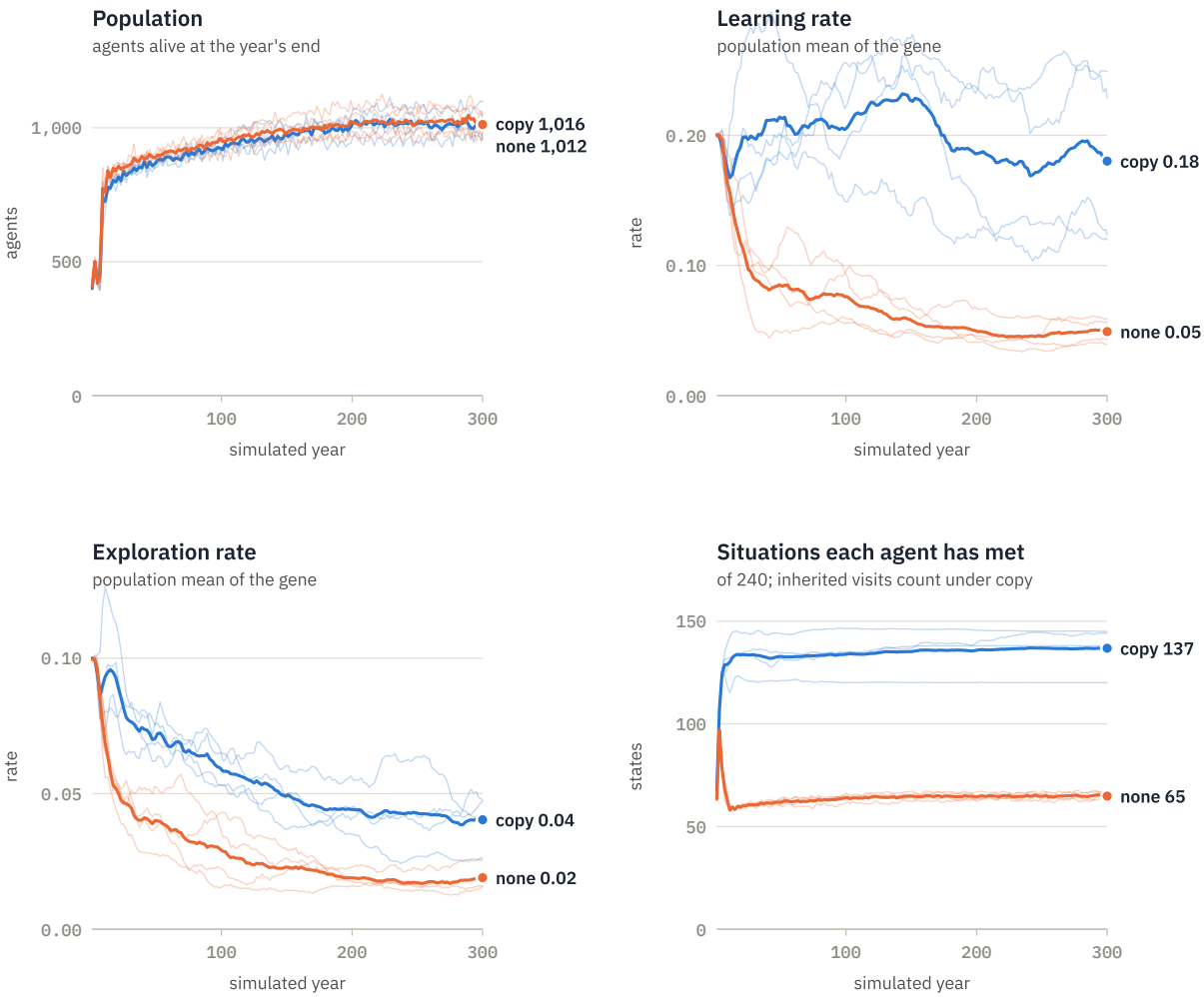
5.5 Disk

Each seed keeps one checkpoint of about 39 MB, overwritten in place (almost all of it is the 2,000 agent slots' Q-tables), and a log that grows by about 140 bytes a simulated year. Sixteen seeds take about 650 MB whatever the run length, plus their logs. To keep a snapshot of a world at some moment, copy its `checkpoint.pkl` somewhere else; it compresses to about a tenth with `gzip`. Checkpoints are Python pickles, and opening a pickle can run code, so only load checkpoints you made yourself.

6 The first experiment: should learning be inherited?

The one design choice river world forces on you is what a child gets from its parent's learning. The first experiment compares the two ends: `copy`, where a child starts with its parent's Q-table, and `none`, where it starts blank and inherits only the genes that set how it learns. Everything else was left at the defaults. Each condition ran four seeds for 300 simulated years on the laptop, eight processes at once, about 16 minutes in all.

```
./run_seeds.sh 4 300 inherit-copy
./run_seeds.sh 4 300 inherit-none --inherit none
python3 summarise.py runs/inherit-copy runs/inherit-none
```



Bold lines: the mean of four seeds; faint lines: each seed. copy = a child inherits its parent's Q-table; none = it starts blank.

Figure 3. The two conditions over 300 simulated years. Population ends the same, but the genes for learning go different ways: when learning is not inherited, evolution turns the learning rate and exploration down. Bold lines are the mean of four seeds; faint lines are the individual seeds.

Table 4. The last 50 years of each run, mean of four seeds (± standard error), with Welch's *t*.

MEASURE	COPY	NONE	*T*
Population	1,012 ± 24	1,024 ± 27	−0.3
Learning rate (gene, started at 0.2)	0.186 ± 0.033	0.049 ± 0.005	4.1
Exploration rate (gene, started at 0.1)	0.041 ± 0.005	0.018 ± 0.002	4.0
Situations met (of 240)	137 ± 6	65 ± 1	12.3
Deepest lineage (generations)	116 ± 7	77 ± 6	4.3
Signal entropy (bits; 2.0 = random)	1.961 ± 0.006	1.998 ± 0.000	−5.9
Behaviour diversity (`ecology_entropy` , bits)	6.07 ± 0.17	4.30 ± 0.17	7.4
Difference between the banks' signals (`dialect_jsd`)	0.0006	0.0001	5.6
Lifespan (gene, started at 1,200 ticks)	3,178 ± 257	3,213 ± 313	−0.1

6.1 What happened

Both worlds filled up the same way. Population rose from 300 to about 700 within ten years and settled near 1,000 in both conditions; the energy budget, not the inheritance rule, decides how many agents the river world feeds. Both populations also evolved the same strategy: the energy needed to reproduce fell from 70 to about 21, close to its floor of 20, and lifespans stretched from 1,200 to about 3,200 ticks. Evolution chose many small agents that split as soon as they can and then live long, hungry lives: the mean agent holds about half a unit of energy. Neither of those results was designed in.

The genes for learning went different ways. When children inherit their parents' Q-tables, the learning rate stays near where it started (0.19) and exploration falls slowly, from 0.10 to 0.04. When every child starts blank, evolution turns learning down: the learning rate falls steadily to 0.05 and exploration to 0.02. All four seeds agree. One reading is that a blank Q-table learned over a short, hungry life is mostly noise, and that noise costs energy, so selection quietens it; whereas an inherited table is worth updating carefully. That is the Baldwin effect's question, how learning and evolution shape each other [1], answered here in one specific way for one specific world. It is a result to test on sixteen seeds and longer runs (Section 7, E2), not a law.

Inherited learning carries more of the world. Agents under `copy` had met 137 of the 240 situations, twice as many as under `none`, because each table carries its ancestors' experience. They also kept a more varied population (6.1 bits of behaviour diversity against 4.3) and their lineages ran deeper.

No dialects yet. Signals stayed close to random in both conditions (entropy 1.96 and 2.00 bits out of 2), and the two banks used them almost identically: under `copy`, `dialect_jsd` peaked at 0.005 around year 25 and ended below 0.001. With three fords and about 300 crossings a year, 300 years is too short and too well connected for the banks to drift apart. That is exactly what Section 7's experiment E3 is for, on the desktop's sixteen cores and over thousands of years.

> **Note:** Four seeds a side is enough to see the large effects here (|t| above 4), but not to trust the small ones. The desktop runs sixteen seeds in the time the laptop ran four; run E1 and E2 there before building on any of this.

7 Experiments to run next

Each experiment below is one change against the defaults, run on 16 seeds. Before you start one, write your prediction in a file next to the runs; the paper's first habit applies here more than anywhere: emergence only means something against a prediction.

Table 5. Six experiments for the desktop, each a single change.

#	QUESTION	COMMAND (16 SEEDS EACH)	READ THESE COLUMNS
E1	Does handing learning down help?	`./run_seeds.sh 16 2000 copy` and `./run_seeds.sh 16 2000 none --inherit none` (add `half` as a third)	`population`, `q_states`, `policy_complexity`
E2	Does evolution tune how agents learn (the Baldwin effect [1])?	The same two conditions as E1	`gene_alpha`, `gene_eps`: do they drift apart between conditions?
E3	Is the river a barrier to culture?	Defaults, 5,000 years	`dialect_jsd`, `crossings`, `pop_west`, `pop_east`
E4	Does disturbance keep the world interesting?	`./run_seeds.sh 16 2000 calm --no-events` against the defaults	`novelty`, `ecology_entropy`, `change`
E5	Starvation or crowding?	`--energy 20000`, the default 30,000, and `--energy 45000`	`population`, `gene_repro`, `gene_lifespan`, `mean_energy`
E6	Does a bigger world sustain more variety?	`--size 128 --energy 53000` (the same energy per cell)	`archive_size`, `ecology_entropy`, `dialect_jsd`

Two experiments need a one-line edit rather than an option, and the code is short enough to make it safely. To change how many fords there are, edit the `self.fords = [...]` line in `World.__init__`; to make the river a wall, give it no fords at all. Make the edit in a copy of the folder, so that the original runs keep resuming with the code they started with.

> **Caution:** A change to the code applies to every world that resumes under it. Never edit `river_world.py` under a running experiment: copy the folder, change the copy, start fresh worlds.

7.1 The three habits, applied

1. **Predict first.** For E3, for example: "with three fords, `dialect_jsd` stays under 0.05; with one ford it climbs above 0.2 within 2,000 years." A number you can be wrong about.
2. **Change one thing, run sixteen seeds.** `summarise.py runs/A runs/B` prints a Welch *t* for every column. With 16 seeds a side, |t| above about 2 is worth a closer look; below that, the difference is within what the seeds do on their own.
3. **Keep everything.** Copy a seed's `checkpoint.pkl` when something odd happens, and take a `snapshot.py` map. The interesting moment in a long run is usually found afterwards, in the log.

7.2 Measuring open-endedness, honestly

The log carries simple versions of the four MODES measures (change, novelty, complexity and ecology) [2]. They are deliberately crude: each agent is summarised by a six-part behaviour descriptor (learning-rate band, exploration band, reproduction band, favourite signal, mobility band and bank), and `novelty` counts descriptors never seen before in that run. The descriptor space has 2,048 cells, so novelty is bound to fall to zero as a run fills it; that is a property of the measure, not proof that the world has stopped producing anything. Whether any of this counts as open-ended depends on the observer [3], and here the observer is these six numbers. Richer descriptors are the obvious first improvement.

8 Reading the log

Each run writes one row a simulated year to `log.csv`. Open it in any spreadsheet, or read it with `summarise.py`.

Table 6. The columns of `log.csv`.

COLUMN	MEANING
`year`	Simulated years since the world was created (400 ticks each)
`population`, `pop_west`, `pop_east`	Agents alive at the year's end, in total and on each bank
`crossings`	Times agents changed bank during the year (only possible at fords)
`mean_energy`	Mean energy of a living agent (out of 150)
`nutrients`, `detritus`, `river_load`	Energy in each pool; with the agents' bodies they always sum to the total
`max_generation`	The deepest lineage alive: generations since the first 300 agents
`gene_lifespan`, `gene_repro`, `gene_mutation`, `gene_alpha`, `gene_eps`	Population means of the five genes: lifespan in ticks, energy to reproduce, mutation scale, learning rate, exploration rate
`q_states`	Mean number of the 240 situations an agent's Q-table has visited (inherited visits count under `--inherit copy`)
`policy_complexity`	Mean number of different actions an agent would choose across the situations it has visited (1 to 20)
`signal_entropy`	Entropy of all signals used in the year, in bits (2.0 means all four equally often)
`dialect_jsd`	Jensen-Shannon divergence between the two banks' signal use, in bits (0 means identical, 1 means disjoint)
`novelty`	Behaviour descriptors seen for the first time this year
`ecology_entropy`	Entropy of the population across behaviour descriptors, in bits
`change`	How far the mean genome has moved in ten years, in units of each gene's mutation scale
`archive_size`	Distinct behaviour descriptors seen since the world began (at most 2,048)
`energy_drift`	Total energy minus 30,000 at the year's end: rounding error only
`event`	Floods, droughts and fires that started during the year

9 Using the RTX 3070

River world never touches the GPU, so the card is free for the two GPU steps of the research paper's eight-week path: fast JAX environments, and a local language model. Give each its own virtual environment.

9.1 JAX on the GPU

The JAX build has to match the driver that Section 3.3 reported. `jax[cuda13]` needs driver 580 or newer, and `jax[cuda12]` needs 525 or newer; the RTX 3070 (Ampere) is supported by both.

```
python3 -m venv ~/venvs/jax && source ~/venvs/jax/bin/activate && pip install -U pip
pip install -U "jax[cuda13]"        # driver 580 or newer; otherwise: pip install -U "jax[cuda12]"
python3 -c "import jax; print(jax.devices())"          # should list a CUDA device, not only the CPU
pip install jaxmarl craftax
```

JaxMARL [4] and Craftax [5] run whole populations of environments on the GPU at once; follow each project's README for a first training run. With 8 GB of memory, start with their default batch sizes and halve them if JAX reports that it is out of memory.

9.2 A local language model

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull qwen3.5:4b                 # 3.3 GB; qwen3.5:2b is 2.7 GB
ollama run qwen3.5:4b "In one sentence, what is a ford?"
```

Qwen3.5's 4B model fits in the 3070's 8 GB with room for a working context; the 9B model (6.6 GB) fits too, but leaves little room [6, 7]. Watch `nvidia-smi` while it answers.

9.3 Why language models cannot run river world's agents

A river world of 800 agents takes 800 decisions a tick and 320,000 a simulated year. Even at ten model calls a second, one simulated year would take about nine hours, and a 1,000-year run about a year. The Q-learners decide in microseconds. Language-model agents belong in a different design: a handful of them, observing the river world or advising one bank, while the population underneath stays cheap. That is step 7 of the research paper's eight-week path, and its Section 11 checklist applies the moment a language model is in the loop: log every prompt, and keep the model's access to the world read-only until you know what it does.

10 Before a week-long run

1. **Prediction written down** in `runs/<name>/PREDICTION.md`, with numbers.
2. **Smoke test passed** on this machine today (Section 4), including the resume check.
3. **One change per condition**, each condition in its own folder, 16 seeds each.
4. **Launcher inside** `tmux` (Section 5.4), so a closed terminal does not stop it.
5. **Temperatures watched for the first hour** (Section 5.3); `JOBS=12` if the CPU runs hot.
6. **Disk checked:** `df -h ~` shows at least 2 GB free for 16 seeds.
7. **The code frozen:** no edits to `river_world.py` while its worlds are running (Section 7).
8. **Summarise daily:** `python3 summarise.py runs/A runs/B` takes a second and changes nothing.
9. **Snapshot weekly,** or whenever a column does something odd: `python3 snapshot.py runs/A/seed-3`.
10. **Copy any checkpoint worth keeping** before the run overwrites it.

11. **After a reboot,** run the same `./run_seeds.sh` line again: every seed resumes.
12. **At the end,** keep the logs, the summaries and the prediction together, and write one paragraph on what was right and what was wrong.

Appendix A: Parameters

All of these are constants or options in `river_world.py`. Options can be set on the command line; constants need an edit (in a copy, Section 7).

Table A1. River world's parameters and their defaults.

PARAMETER	DEFAULT	WHERE
World size	96 × 96 cells, wrapping north to south, walled east and west	`--size`
Total energy	30,000 units, conserved	`--energy`
Agent capacity	2,000 at once	`--capacity`
Starting population	300 agents, 40 energy each, on random land cells	`World.__init__`
Terrain	rock 5% of cells, clearings 22%, forest the rest; the river is carved through them	`World.__init__`
River	3 cells wide; centre swings up to about 11 cells either side; fords at rows 16, 48 and 80, three rows deep	`World.__init__`
Time	day 20 ticks (night is the last 8); season 100 ticks; year 400 ticks	`DAY`, `SEASON`, `YEAR`
Costs	rest 0.5, move 0.9, ×1.25 at night, +0.05 to send a signal	`World.step`
Eating	up to 4 a tick; at most 150 held	`FEED`, `MAX_E`
Decomposition	5% (forest), 3.5% (clearing), 2% (ford) of detritus a tick; ×1.2 spring, ×1.0 summer, ×0.8 autumn, ×0.4 winter	`World.step_environment`
Diffusion	2% of a cell's nutrients a tick to each land neighbour	`World.step_environment`
River	erosion 0.2% of bank nutrients a tick; 20% of each row's load moves one row south a tick; 1% of the load settles as silt a tick, ×2.5 in spring	`World.step_environment`
Flood	25% chance each spring; 40 ticks of 6× silt and 3× erosion	`World.maybe_event`
Drought	15% chance each summer; 100 ticks of the river at a quarter of its flow and silt	`World.maybe_event`
Fire	12% chance at the start of summer and of autumn; radius 6; nutrients become detritus; agents in it lose half their energy	`World.maybe_event`
Learning	tabular Q-learning, 240 states × 20 actions, discount 0.95, reward = change in energy, random tie-breaks	`World.step`
Starting genes	lifespan 1,200 ticks, reproduce above 70 energy, mutation scale 0.05, learning rate 0.2, exploration 0.1	`World.__init__`
Mutation	each gene moves by a normal step × (mutation scale ÷ 0.05) × 80 ticks, 8 energy, 0.01, 0.05 and 0.03 respectively, within fixed limits	`G_SCALE`, `G_MIN`, `G_MAX`
Inheritance	the parent's Q-table is copied to the child	`--inherit copy`, `half` or `none`
Checkpoint	every 10 years, and on exit	`--save-every`

Appendix B: How the kit was tested

Everything below was run on the laptop before this paper was written, and every test passed in its final form.

- **Energy.** Conservation is asserted after every tick in every run. In the eight 300-year runs of Section 6 the largest end-of-year drift was 3.1×10^{-8} energy units, about one part in 10^{12} of the total.
- **Resume.** A run stopped and resumed produced a log byte-identical to an uninterrupted run in four cases: stopped at a year boundary; stopped part-way through a year; stopped with Ctrl-C at year 8 of 20; and killed outright with `kill -9` after its last checkpoint, where the resumed run trimmed the one log row written after the checkpoint and carried on.
- **Symmetry.** A world and its exact mirror image, left for right, were run side by side. The first version of the code favoured the east bank, by between three and seven to one, because untried actions were chosen in index order and east comes before west. Random tie-breaking removed the bias: which bank leads now varies with the seed and the mirror, as it should.
- **Review.** A line-by-line review found seven defects in the first version, all fixed and re-tested: an interrupted run skipped the seasonal event roll and could not resume exactly; the signal colouring of maps was always blank; maps of extinct worlds crashed; one failed seed stopped the summary; extinct seeds were hidden in the summary; a log row still being written crashed it; and a hard kill could leave duplicate rows in the log.
- **Tools.** `run_seeds.sh`, `summarise.py` and `snapshot.py` were run on real and on deliberately extinct worlds.

References

1. Hinton, G. E. and Nowlan, S. J. (1987). How learning can guide evolution. Complex Systems 1(3), 495-502. https://www.complex-systems.com/abstracts/v01_i03_a06/
2. Dolson, E., Vostinar, A., Wiser, M. and Ofria, C. (2019). The MODES toolbox: measurements of open-ended dynamics in evolving systems. Artificial Life 25(1), 50-73. https://doi.org/10.1162/artl_a_00280
3. Hughes, E. et al. (2024). Position: Open-Endedness is Essential for Artificial Superhuman Intelligence. ICML 2024 (oral), PMLR 235, 20597-20616. arXiv:2406.04268. https://arxiv.org/abs/2406.04268
4. Rutherford, A. et al. (2024). No Regrets: Investigating and Improving Regret Approximations for Curriculum Discovery. NeurIPS 2024. arXiv:2408.15099. https://arxiv.org/abs/2408.15099
5. Matthews, M. et al. (2024). Kinetix: Investigating the Training of General Agents through Open-Ended Physics-Based Control Tasks. ICLR 2025 (oral). arXiv:2410.23208. https://arxiv.org/abs/2410.23208
6. Qwen team (2026). Qwen3.5 models on Hugging Face, February 2026. https://huggingface.co/Qwen
7. Ollama model library, read 10 October 2026. https://ollama.com/library/qwen3.5